



SOLUTIONS FOR VIRTUAL TEST DRIVING

CarMaker Python API

User's Guide Version 15.0



The information in this document is furnished for informational use only, may be revised from time to time, and should not be construed as a commitment by IPG Automotive GmbH and its affiliates. IPG Automotive GmbH and its affiliates assumes no responsibility or liability for any errors or inaccuracies that may appear in this document.

This document contains proprietary and copyrighted information and may not be copied, reproduced, translated, or reduced to any electronic medium without prior consent, in writing, from IPG Automotive GmbH.

For better readability, the masculine form is used for personal designations and words referring to persons in this documentation. In the interests of equal treatment, the corresponding terms apply to all genders as we are an inclusive and diverse organization and we recognize the diversity. The abbreviated form of language is for editorial reasons only and does not imply any judgment.

© 1999 - 2025 by IPG Automotive GmbH – www.ipg-automotive.com
All rights reserved.

Fail Safe Tester, IPGCar, IPGControl, IPGDriver, IPGEngine, IPGGraph, IPGKinematics, IPGLock, IPGMotorcycle, IPGMovie, IPGRoad, IPGRoaddata, IPGTire, IPGTrailer, IPGTruck, RealtimeMaker, Xpack RTS are trademarks of the IPG Automotive GmbH.

CarMaker, TruckMaker, MotorcycleMaker, MESA VERDE, VIRTO, IPG, IPG Automotive are registered trademarks of IPG Automotive GmbH.

All other product names are trademarks of their respective companies.

Affiliate means any corporate body of which IPG Automotive GmbH has a majority stake.

All CarMaker variants use several open source components. Please refer to the files FOSS-Licenses.html and FOSS-Licenses.pdf in the /doc folder of the CarMaker installation directory for more details.

CarMaker variants use additional components of proprietary third-party software. The corresponding end-user license agreements (EULA) can be found in directory doc/EULA-3rd-Party of each CarMaker installation.

RealtimeMaker uses several open source components. Please refer to the files FOSS-Licenses.html and FOSS-Licenses.pdf in the /doc folder of the RealtimeMaker installation directory for more details.

Movie NX: portions of this software Copyright © 2025 Unigine. All rights reserved. Used under license. Qt licensed under the Lesser General Public License (LGPL) version 3.

To obtain the Qt source code please contact support@ipg-automotive.com.

Movie NX uses several open source components. Please refer to the files FOSS-Licenses.html and FOSS-Licenses.pdf in the /doc folder of the Movie NX installation directory for more details.

IPGControl uses several open source components. Please refer to the files FOSS-Licenses.html and FOSS-Licenses.pdf in the /doc folder of the IPGControl installation directory for more details.

Contents

1	Overview	5
1.1	About this Add-on	5
1.2	Installing the AddOn	6
1.3	Installing the wheels	6
1.4	First Use	7
2	XIL API	9
2.1	Introduction	9
2.1.1	Additional Files	10
2.2	XIL API by Example	10
2.2.1	Example 1_ReadWrite	10
2.2.2	Example 2_BasicCapturing	12
2.2.3	Example 3_CapturingWithDurationWatcher	13
2.2.4	Example 4_CapturingWithConditionWatcher	13
2.2.5	Example 5_CaptureToFile	14
2.2.6	Example 6_MultiFetch	14
2.2.7	Example 7_ConfigurationParameters	15
2.3	CarMaker XIL API on Third-Party HIL Systems	16
2.3.1	dSPACE SCALEXIO	16
2.3.2	NI Linux Real-Time	17
2.4	XIL API Function Reference	18
2.4.1	TestbenchFactory	18
2.4.2	Testbench	19

2.4.3	MAPort	21
2.4.4	Capture	28
2.4.5	CaptureResult	31

Chapter 1

Overview

1.1 About this Add-on

This add-on package can be installed with the IPG Installer into a matching CarMaker installation.

It contains the CarMaker Python API for runtime automatization in Python.

The CarMaker Python API consists of the following Python packages:

- cmapi
- apoc
- infofiles
- ASAM/XIL

The first three packages provide the CarMaker Python API core.

The ASAM/XIL package contains an implementation of the ASAM/XIL API for CarMaker.

The CarMaker Python API core offers an alternative script level interface to the CarMaker simulation framework, using the Python programming language. In addition to being more accessible to users with experience in Python, it also deploys a more object-oriented approach, utilizing the full flexibility of the Python programming language.

By building upon the Python framework, the user of the CarMaker Python API benefits from the ease of use and comprehensiveness of the Python programming language, with extensive community support and availability of third-party libraries for all possible applications.

CarMaker Python API should not be understood as a mere port of Tcl Script Control commands to Python. Script Control is intended to offer an additional command line interface to the CarMaker GUI and as a result mainly mirrors the GUI workflow, while sacrificing flexibility. CarMaker Python API on the other hand was developed explicitly as a pure script level interface for CarMaker runtime automation, with a focus on adaptability and stand-alone usage. To use CarMaker Python API no CarMaker GUI is required. The highly imperative paradigm of Script Control was replaced by a more object-oriented approach. CarMaker Python API ranges from mid-level to low-level, allowing to operate on a much lower level than Script Control.

CarMaker Python API features include:

- Setup and parametrization of Test Runs
- Control of the running simulation
- Management of CarMaker Interface Toolbox applications
- Tools to easily extend the automation scope up to multiple simulations running in parallel

For information about the ASAM/XIL API, refer to [section 'XIL API'](#).

1.2 Installing the AddOn

To install a CarMaker Python API add-on package, follow the regular installation process for CarMaker add-ons using IPG Installer. Supported versions can be identified from the add-on package's name:

`CarMaker-Python-API-linux64-15.0-py3.9..3.13.txz`

The first version number is the add-on's version, which must share its major version with a CarMaker installation. The minor and patch level version numbering of CarMaker Python API add-on packages and CarMaker installation packages do not have an impact on compatibility and are independent of one another. The following version range `py3.9..3.13` indicates that this add-on contains Python packages compatible with Python versions 3.9, 3.10, 3.11, 3.12 and 3.13.

The IPG Installer selects only compatible add-ons for a given CarMaker installation. After the installation the Python packages can be found in

`<IPGHOME>/carmaker/<ARCH>-<VER>/Python`

and the example script files can be found in

`<IPGHOME>/carmaker/<ARCH>-<VER>/Examples/Python`

1.3 Installing the wheels

The CarMaker Python API add-on contains wheel packages for a range of Python versions.

Wheels for the modules *apoc* and *infofiles* are only compatible with a Python interpreter that matches their version. The *cmapi* package is built with the `PY_LIMITED_API` option and is compatible with the supported Python versions. The ASAM/XIL package is a precompiled package (also built with the `PY_LIMITED_API` option) that is currently not provided as a wheel (see next chapter). The ASAM/XIL package is also compatible with the range of supported Python versions.

Before installing the wheels, the Python version needs to be checked. Run

```
python3 -V
```

and compare the output string of that command with the Python versions mentioned in the names of the *apoc* and *infofiles* wheels. The *cmapi* wheel is compatible with the whole range of supported Python versions. To install a wheel, run

```
pip3 install package.whl
```

1.4 First Use

If no Python interpreter is installed yet, it can be downloaded here: <https://www.python.org/downloads/>.



To check if an installed Python interpreter matches an installed add-on's version, run

```
python3 -V
```

On Windows, the Python executable name is different.



CarMaker Python API contains Python wheels for the modules *apoc* and *infofiles* that are compiled for a specific Python version exclusively. Trying to install these from an interpreter with a different release version is not possible.

When the Python interpreter is up and running, and the wheels are installed, the *cmapi* package can be imported. To work with the ASAM/XIL package, that is currently not provided as a wheel, the path to the package has to be added. This can be done in two ways:

- Setting the path inside the script before the module is imported.
To do so, use the following commands:

```
1: import sys
2: sys.path.append("C:/IPG/carmaker/win64-15.0/Python")
```

- Adding the CarMaker Python API to the PYTHONPATH environment variable



- **Linux:**

In the shell, use the following command:

```
export PYTHONPATH=/opt/IPG/carmaker/linux64-15.0/Python/
python<PYTHONVER> (m) :$PYTHONPATH
```



- **Windows:**

In the command line, use the following command:

```
set PYTHONPATH=C:/IPG/carmaker/win64-15.0/Python/
python<PYTHONVER>;%PYTHONPATH%
```

The CarMaker Python API is configured to use the CarMaker installation directory it resides in as the default data pool. Furthermore, Interface Toolbox applications and the CarMaker simulation program in the CarMaker installation may be executed by some CarMaker Python API function calls.



To switch to another Python version, install the wheels and/or set the path to the directory containing the desired version. It is generally recommended to use virtual environments for a clean installation.

To use CarMaker Python API from a Python script or Python console, it must be imported by calling

```
1: import cmapi
```

Getting extended diagnostic output

To get extended diagnostic output of CarMaker Python API functions, there exists a logger, which can be turned to verbose by either activating it programmatically:

```
1: cmapi.logger.setLevel("DEBUG")
```

or by setting an environment variable in the shell from which the Python interpreter is called

```
1: CMAPIDEBUG=DEBUG
```

When everything is set up, open the CarMaker Python API example script `runtime_minimal_example.py` in the CarMaker installation directory in the *Examples/Python* folder in an editor.

In the line containing `project_path=Path("path/to/CM/project")`, make `project_path` pointing to an existing project directory.

Run `runtime_minimal_example.py` with the following command:

```
python3 runtime_minimal_example.py
```

If a CarMaker Python API add-on is installed for CarMaker, its documentation is available in several ways:

- Via CarMaker GUI
To open the documentation, in the CarMaker GUI, click *Help > Additional Documents > CarMaker Python API*.
- Via file browser
The documentation is stored in the CarMaker's installation directory in the `/doc/CMAPI/` folder. Open the file `index.html` with a web browser.

The Python API documentation contains information about all API classes and functions and a detailed introduction section for first-time users.



The *Examples/Python* folder also provides the script `userconfig.py` which was located in the `cmapi` module folder in previous releases. This script contains a Python dictionary that is read when the `cmapi` module is imported and allows to modify parameters and settings. This script is not mandatory and can be omitted if no changes are required.

Chapter 2
XIL API

2.1 Introduction

The ASAM XIL specification was developed to enable easy and reproducible testing with different combinations of hardware and software tools. The standard consists of two layers, the Testbench API and the Framework API. CarMaker contains an implementation of the ModelAccess Port (MAPort) to access to the simulation model, read and write parameters and capture data from the simulation program.

To control CarMaker using the XIL Testbench API, a working Python installation is needed on the system. The currently supported Python version is listed in the Release Notes, [section 'Python'](#).

Using examples with CarMaker

To use the examples delivered with CarMaker, the *matplotlib* module needs to be installed, e.g. by using *pip installer*:

```
pip install matplotlib
```

Using CarMaker Testbench in individual Python scripts

To use the CarMaker Testbench in individual Python scripts, add the path to the *<CarMakerInstall>/Python/python<PYTHONVER>* directory to the Pythonpath. Do this by adding this path to the PYTHONPATH environment variable or by adding the lines:

```
1: import sys
2: sys.path.append("C:/IPG/carmaker/win64-15.0/Python")
```

to your script.

After that, run scripts using XIL API functions by starting the script in any Python interpreter of the supported Python versions.



To use CarMaker on a National Instruments HIL system, install *pythonnet* and import it at the beginning of the script, using:

```
pip install pythonnet
```

and

```
import clr
```



To use dSPACE SCALEXIO systems with XIL API functions, install *telnetlib3*

```
pip install telnetlib3
```

2.1.1 Additional Files

Before using the provided functionality, two additional files need to be set up on the system. The first is the Manifest file that contains information for the TestbenchFactory. The TestbenchFactory is needed to create Testbench objects in the code.

Copy the Manifest file to the `%Program Data%` folder (e.g.: `C:/Program Data/ASAM/XIL/Implementation`) on Windows, or to `/opt/ASAM/XIL/Implementation` on unix systems. For testing or development purposes, it is also possible to copy the Manifest file into the same folder as the Python script. The *Examples* folder contains a valid IMF file to find the Testbench of the current CarMaker version.

The second file needed is a configuration XML file. The *Examples/Python* folder contains some examples for a configuration file. Parameters are explained in detail in [section '<MAPort>.LoadConfiguration'](#).



Make sure to replace all placeholder variables in the examples with valid paths on your system if you want to use them in your script.

2.2 XIL API by Example

To give you a quick overview of the available functionality with XIL API for CarMaker, the installation provides a couple of small examples in the `<CarMakerInstall>/Examples/Python` folder. This chapter explains the used functionality one by one.

2.2.1 Example 1_ReadWrite

Any script using XIL API functionality has to start by importing a TestbenchFactory capable of reading IMF Manifest files and creating Testbench instances. CarMaker provides an implementation of such a TestbenchFactory in the Python directory of the CarMaker installation. You can use this implementation to search for Python implementations of XIL Testbenches. The first lines of code in the example show how to add the path to the Python folder to your Pythonpath and import the TestbenchFactory module:

Step 1 Add CarMaker installation to your Pythonpath and import basic modules:

```
1: import sys
2: sys.path.append("C:/IPG/carmaker/win64-15.0/Python")
3: from ASAM.XIL.Implementation.Testbench import TestbenchFactory
4: from ASAM.XIL.Interfaces.Testbench.MAPort.Enum.MAPortState import MAPortState
5: from ASAM.XIL.Interfaces.Testbench.Common.Error.TestbenchPortException import
   TestbenchPortException
```

The example uses the *Config.xml* file to configure the Testbench later on. Make sure to replace the `%YourProject%` entry in the *Config.xml* file with the path to a valid project directory before running the script.

```
6: MAPortConfigFile = "Config.xml"
```

Step 2 Instantiate a TestbenchFactory and create the Testbench from it:

```
7: MyTestbenchFactory = TestbenchFactory()
8: MyTestbench = MyTestbenchFactory.CreateVendorSpecificTestBench("IPG", "CarMaker",
   "15.0")
```

Step 3 Use the Testbench object to create additional Factory classes you need to use. In this example, only a MAPort is used for the basic read and write functionality:

```
9: MyMAPortFactory = MyTestbench.MAPortFactory
10: DemoMAPort = MyMAPortFactory.CreateMAPort("DemoMAPort")
```

Step 4 A MAPort object needs to be configured before it can be used. To do this, the two functions `<MAPort>.LoadConfiguration()` and `<MAPort>.Configure()` need to be called in the following order:

```
11: DemoMAPortConfig = DemoMAPort.LoadConfiguration(MAPortConfigFile)
12: DemoMAPort.Configure(DemoMAPortConfig, False)
```

Note, that we pass the previously defined *Config.xml* file to the *LoadConfiguration()* function. This loads the parameters defined in the XML file and passes them to the CarMaker process when it is started up in the *Configure()* function.

Step 5 After the MAPort has been successfully configured, a simulation can be started to interact with it. A Test Run of your choice can be passed directly to the `<MAPort>.StartSimulation()` function.

Before starting the simulation, make sure that there is no simulation already running. The MAPort indicates a running simulation with its State set to *eSIMULATION_RUNNING*. For a description of all MAPort states see [section 2.4.3 'MAPort'](#):

```
13: if DemoMAPort.State is not MAPortState.eSIMULATION_RUNNING:
14:     DemoMAPort.StartSimulation("Examples/BasicFunctions/Driver/BackAndForth")
```

Step 6 When the simulation has started, variables can be read and written to and from it using `<MAPort>.Read()` and `<MAPort>.Write()`. In this example, the `<MAPort>.WaitForTime()` convenience function is used to wait for specific simulation times to be reached before reading or writing:

```
15: # Overwrite the Steering angle during simulation at different times
16: DemoMAPort.WaitForTime(10.0, 30.0)
17: # change steering angle
18: print("SteerAng after 10s: " + str(DemoMAPort.Read("DM.Steer.Ang")))
19: print("Setting steer angle to 1 rad for 500ms")
20: DemoMAPort.Write("DM.Steer.Ang", 1.0, 500)
21: DemoMAPort.WaitForTime(10.3, 30.0)
22: print("New SteerAngle after write: " + str(DemoMAPort.Read("DM.Steer.Ang")))
23:
24: DemoMAPort.WaitForTime(20.0, 30.0)
25: # change steering angle without pause
26: print("SteerAngle after 20s: " + str(DemoMAPort.Read("DM.Steer.Ang")))
27: print("Setting steer angle to -2 rad for 500ms")
28: DemoMAPort.Write("DM.Steer.Ang", -2.0, 500)
29: DemoMAPort.WaitForTime(20.3, 30.0)
30: print("New steering angle after write: " + str(DemoMAPort.Read("DM.Steer.Ang")))
```

Step 7 When we are done messing with the vehicles steering, the simulation is stopped (`<MAPort>.StopSimulation()`) and the MAPort that closes the connection to and stops the CarMaker process is disposed:

```
31: # Stop simulation
32: DemoMAPort.StopSimulation()
33:
34: if DemoMAPort != None:
35:     DemoMAPort.Dispose()
36:     DemoMAPort = None
```

2.2.2 Example 2_BasicCapturing

The first lines of all examples are the same as in [section 2.2.1 'Example 1_ReadWrite'](#) since the import of modules and the configuration are always needed to work with the MAPort.

This example focuses on the steps needed to set up a Capture object to collect some data from CarMaker.

Step 1 After the MAPort has been successfully configured, a Capture object can be created and configured to subscribe to some variables in CarMaker:

```
1: # Configure Variables to capture
2: DemoCapture = DemoMAPort.CreateCapture("captureTask")
3: DemoVariableList = ["Car.ax", "Car.ay", "Car.az"]
4: DemoCapture.Variables = DemoVariableList
5: DemoCapture.Downsampling = 50
6: # capture data once the testrun starts
7: DemoCapture.Start()
```

The `<Capture>.Start()` function activates only the Capture. The collection of data starts when the simulation is started and the `CaptureState` is set to `eRUNNING` (see Capture states in [section 2.4.4 'Capture'](#)).

Step 2 When the simulation has started, the Capture collects values of the specified variables with the given downsampling (see `<Capture>.Downsampling()`) for the entire Test Run.

When the simulation has finished (which we can wait for using the `<MAPort>.WaitForSimEnd()` convenience function), the Capture is stopped and the result is retrieved by calling the `<Capture>.Fetch()` function:

```
8: # Wait for the simulation to end and get the CaptureResult
9: DemoMAPort.WaitForSimEnd(120.0)
10: DemoCapture.Stop()
11: Result = DemoCapture.Fetch(True)
```

Step 3 The captured data can be postprocessed in any way. In this example, `matplotlib` is used to display the captured acceleration values over time:

```
12: # Visualization of capture result using matplotlib
13: TimeSignalValue = Result.ExtractSignalValue("Time")
14: XAxisValues = TimeSignalValue.Value
15:
16: AccXSignalValue = Result.ExtractSignalValue("Car.ax")
17: AccXValues = AccXSignalValue.Value
18:
19: AccYSignalValue = Result.ExtractSignalValue("Car.ay")
20: AccYValues = AccYSignalValue.Value
21:
22: AccZSignalValue = Result.ExtractSignalValue("Car.az")
23: AccZValues = AccZSignalValue.Value
24:
25: plt.xlabel("Seconds")
26: plt.ylabel("m/s^2")
27: plt.xlim(min(XAxisValues), max(XAxisValues))
28: plt.plot(XAxisValues, AccXValues, 'r')
29: plt.plot(XAxisValues, AccYValues, 'g')
30: plt.plot(XAxisValues, AccZValues, 'b')
31: # save figure to file
32: plt.savefig('mygraph.png')
33: # or show figure on screen
34: # plt.show()
```

Step 4 Like MAPort objects, dispose Captures if they are no longer needed:

```
35: if DemoCapture != None:
36:     DemoCapture.Dispose()
37:     DemoCapture = None
```

2.2.3 Example 3_CapturingWithDurationWatcher

This example shows how to create Start and Stop Triggers for a Capture object to only capture a specific time span during the simulation.

To create the necessary Watcher objects and delays, *WatcherFactory* and *DurationFactory* are used:

```
1: MyWatcherFactory = MyTestbench.WatcherFactory
2: MyDurationFactory = MyTestbench.DurationFactory
```

In addition to the variables list and downsampling of the capture, a Start (*<Capture>.SetStartTrigger()*) and a Stop Trigger (*<Capture>.SetStopTrigger()*) with an additional delay (*<DurationFactory>.CreateTimeSpanDuration()*) is set:

```
3: # Set DurationWatchers as Start and Stop Triggers
4: DemoStartWatcher = MyWatcherFactory.CreateDurationWatcherByTimeSpan(10.0)
5: StartDelay = MyDurationFactory.CreateTimeSpanDuration(-1.0)
6: DemoCapture.SetStartTrigger(DemoStartWatcher, StartDelay)
7: StopDelay = MyDurationFactory.CreateTimeSpanDuration(1.0)
8: DemoStopWatcher = MyWatcherFactory.CreateDurationWatcherByTimeSpan(30.0)
9: DemoCapture.SetStopTrigger(DemoStopWatcher, StopDelay)
```

This setup results in the capture being started 9 seconds into the simulation and ending after 31 seconds in the simulation. The result is visualized as before in a *matplotlib* plot.

2.2.4 Example 4_CapturingWithConditionWatcher

This example shows the Condition Watcher, another type of Start/Stop Trigger. With the Condition Watcher, the value of a variable in the simulation can be used to trigger start and stop of the Capture.

To create a Condition Watcher object, the *<WatcherFactory>.CreateConditionWatcher()* function of the WatcherFactory is used:

```
1: # ConditionWatcher Definition
2: StartDelay = MyDurationFactory.CreateTimeSpanDuration(-2.0)
3: DemoStartWatcher = MyWatcherFactory.CreateConditionWatcher("Car.ay > 1.5")
4: DemoCapture.SetStartTrigger(DemoStartWatcher, StartDelay)
5: StopDelay = MyDurationFactory.CreateTimeSpanDuration(2.0)
6: DemoStopWatcher = MyWatcherFactory.CreateConditionWatcher("Car.ay < -1.5")
7: DemoCapture.SetStopTrigger(DemoStopWatcher, StopDelay)
```

The syntax used for CarMaker conditions differs slightly from the syntax specified in the XIL API (see section 'Real-time Expressions' in the User's Guide).

2.2.5 Example 5_CaptureToFile

So far, data is captured to memory and postprocessing handled directly in the script. This example shows how to capture data to an MDF file instead.

The file to capture to can be configured in a `CaptureResultMDFWriter` object that can be created using the `CapturingFactory` ([<CapturingFactory>.CreateCaptureResultMDFWriterByFileName\(\)](#)):

```
1: MyCapturingFactory = MyTestbench.CapturingFactory
2: # create mdffilewriter
3: DemoCaptureWriter =
    MyCapturingFactory.CreateCaptureResultMDFWriterByFileName(MDFFile)
```



Existing files with the same name are overwritten by default!

To change this behavior, set the *Overwrite* variable of the *MDFWriter* to *False*:

```
4: DemoCaptureWriter.Overwrite = False
```

Two restrictions apply when capturing to file with *CarMaker*:

- With the Stop Trigger ([<Capture>.SetStopTrigger\(\)](#)), no negative delays can be used.
- The written MDF files are always of version 4.1, since this is the currently supported MDF version in *CarMaker*.

2.2.6 Example 6_MultiFetch

This example shows how to continuously get data from the capture during a simulation by repeatedly calling the [<Capture>.Fetch\(\)](#) function.

After configuring the *Capture* and starting the simulation, the example loops and calls *Fetch()* with the *whenFinished* parameter set to *False* every second:

```
1: # Fetch data multiple times and do something with it
2: # here we just save the data to display later
3: # capture every second of data between start and endcondition
4: count = 0
5: # set simtime to first second after startcondition is triggered
6: simtime = 10.0
7: Results = []
8: # repeatedly call fetch to get data.
9: # CAUTION: avoid processing the data during simulation as this can slow
10: # down the capturing process and lead to loss of data
11: while DemoCapture.State != CaptureState.eFINISHED:
12:     DemoMAPort.WaitForTime(simtime)
13:     # save data for post-processing
14:     Results.append(DemoCapture.Fetch(False))
15:     simtime = simtime + 1.0
16:     # Fetch returns None if Capture hasn't started yet or no data is available
17:     if Results[-1] is None:
18:         del Results[-1]
19:         continue
```

The returned *CaptureResults* are stored in a list for postprocessing later. When the simulation is finished, all of them are plotted.

The last call to *Fetch()* in the loop as well as the *Fetch()* call with *whenFinished* set to *True* after the loop returns the data for the entire capture time frame. This is because the last call in the loop occurring after the Stop Trigger has set the *CaptureState* to *eFINISHED*. In this state, *Fetch()* always returns the entire *CaptureResult* from start to finish.

2.2.7 Example 7_ConfigurationParameters

This example illustrates how to parameterize CarMaker in more detail. The first main difference is that this example uses a configuration XML file that contains all possible parameters that can be set. The *Config_Example.xml* file can be found in the same folder as the examples. By default the script in this example shows how to start a custom executable from the *src* folder in a project on an Xpack4 real-time system.

In the XML file, fill in the placeholder paths with the paths to the project, the custom installation directory and the hostname of the xeno system. In addition, DataPools can be added to the DataPools list:

```
1: <?xml version="1.0" encoding="utf-8"?>
2: <PortConfigurations xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
3:   <MAPortConfig>
4:     <ProjectDir>%YourProject%</ProjectDir>
5:     <Platform>xeno</Platform>
6:     <CMPPath>src/CarMaker.xeno</CMPPath>
7:     <InstallPath>%CustomInstallDir%</InstallPath>
8:     <DataPools>
9:       <DataPool>%AdditionalDataPool%</DataPool>
10:    </DataPools>
11:   </MAPortConfig>
12: </PortConfigurations>
```

In the script, a new vehicle, a trailer and a new set of tires are defined for the *Racetrack_Hockenheim* Test Run using the [*<MAPort>.DownloadParameterSets\(\)*](#) function:

```
1: DemoMAPort.DownloadParameterSets(["Examples/VehicleDynamics/Handling/
   Racetrack_Hockenheim",
2: "Examples/Demo_Audi_R8"
3: "Examples/RT_235_55R18",
4: "Examples/Caravan"])
```

2.3 CarMaker XIL API on Third-Party HIL Systems

Since CarMaker can be run on dSPACE and National Instruments HIL systems, the XIL API Testbench can be configured to start and connect to a CarMaker process running on such a system. To do so, certain parameters need to be changed and scripts adjusted accordingly. This chapter goes through the supported dSPACE and National Instruments HIL systems one by one and explains the necessary adjustments. An example XML configuration file for every type of HIL system can be found in the *Examples/Python* folder in the CarMaker installation directory.

In addition, the following software components need to be installed to enable the download and execution of CarMaker on third-party systems:

- **For dSPACE:**
ControlDesk software containing the *CmdLoader.exe*. Additionally for SCALEXIO projects, *ConfigurationDesk* needs to be installed to build RTA or SDF files.
- **For National Instruments:**
NI VeriStand software and the corresponding NI XIL Testbench DLLs that can be found in the NI VeriStand installation directory.

2.3.1 dSPACE SCALEXIO

To run CarMaker on a dSPACE SCALEXIO system, the following parameters need to be set in a *Config.xml*:

- Set the `Platform` parameter to *dsrt*.
- Set the `Hostname` parameter to the name the SCALEXIO system was registered under in the Control/ConfigurationDesk software, e.g. *SCALEXIO*.
- Set the `CMPath` parameter to contain a path to a valid RTA or SDF file that can be downloaded onto a SCALEXIO real-time system. These files can be created using the *ConfigurationDesk* software and can be found in the *Build Results* folder of the ConfigurationDesk project.



The dSPACE version used to compile the file needs to be the same as the currently active dSPACE version. If the version is not the same, CmdLoader is not able to download the file.

As mentioned in [section 2.1 'Introduction'](#), the communication with CarMaker on SCALEXIO also requires *telnetlib3*. Note that as of Python 3.13, the official *telnetlib* module is removed. To install *telnetlib3* using pip, use:

```
pip install telnetlib3
```


2.3.2 NI Linux Real-Time

To run CarMaker on a National Instruments Linux real-time system, the following parameters need to be set in a *Config.xml*:

- Set the `Platform` parameter to *nilrt*.
- Set the `Hostname` parameter the IP or DNS name of the Linux real-time system.
- Set the additional `NIConfiguration` parameter to contain the path to a valid *Config.xml* file for an NI Testbench. An example for such a configuration file is contained in the *Examples/Python* folder.
If this path is relative, it is treated as relative to the CarMaker project directory specified in the same configuration file.

To be able to access NI XIL Testbench functionality, the *pythonnet* module needs to be installed using:

```
pip install pythonnet
```

The *pythonnet* module needs to be imported at the beginning of any script used to run an NI/CarMaker project:

```
import clr
```



Linux real-time systems require additional setup steps to run CarMaker. For detailed instructions see [section 1.9 'Using CarMaker HIL with National Instruments Hardware'](#).

2.4 XIL API Function Reference

This chapter gives an overview of all implemented classes and their functions and parameters in the CarMaker Testbench.

2.4.1 TestbenchFactory

The TestbenchFactory class provides two functions to find PyTestbenchImplementations in XIL Manifest files. These are usually used at the beginning of a script to instantiate a Testbench of a certain product with a specific version. The TestbenchFactory class can be imported in the script and then instantiate a TestbenchFactory directly:

```
1: from ASAM.XIL.Implementation.Testbench import TestbenchFactory
2: testbenchfactory = TestbenchFactory()
```

The following functions can be called as members of the TestbenchFactory object:

<TestbenchFactory>.CreateVendorSpecificTestbench

Syntax

CreateVendorSpecificTestbench(string *vendorName*, string *productName*, string *productVersion*)

Description

Returns an object of the Testbench specified by the *vendor* and *productName* as well as the version.

Example

```
TestbenchFactory.CreateVendorSpecificTestbench("IPG", "CarMaker", "15.0")
```

<TestbenchFactory>.GetAvailableTestbenches

Syntax

GetAvailableTestbenches()

Description

Returns a list of all available PyTestbenchImplementations in all Manifest files found on the system.

Example

```
TestbenchFactory.GetAvailableTestbenches()
```

2.4.2 Testbench

The main Testbench class gives access to additional Factory classes for different functionalities. The CarMaker Testbench implements the following Factory classes:

MAPortFactory

Creates MAPort objects that correspond to a single CarMaker process. Also enables to control and exchange data with the simulation program. The MAPortFactory can be accessed directly from the Testbench object:

```
maportfactory = Testbench.MAPortFactory
```

<MAPortFactory>.CreateMAPort

Syntax

CreateMAPort(string *name*)

Description

Creates a MAPort object with a given *name* as identifier.

Example

```
MAPort = MAPortFactory.CreateMAPort("DemoMAPort")
```

WatcherFactory

Creates Watcher objects that can be used to react to events during simulation, for example when a given simulation time is reached or when certain variables reach specified thresholds. Watcher objects can be passed to Capture objects to control the conditions for capturing. The WatcherFactory can be accessed directly from the Testbench object:

```
watcherfactory = Testbench.WatcherFactory
```

<WatcherFactory>.CreateDurationWatcherByTimeSpan

Syntax

CreateDurationWatcherByTimeSpan(float *timespan*)

Description

Creates a Watcher object that monitors simulation time and triggers when the simulation time reaches the given value. A Watcher object can be used to start ([<Capture>.SetStartTrigger\(\)](#)) or stop ([<Capture>.SetStopTrigger\(\)](#)) a capture.

Example

```
Watcher = WatcherFactory.CreateDurationWatcherByTimeSpan(10.0)
```

<WatcherFactory>.CreateConditionWatcher

Syntax

CreateConditionWatcher(string *rtexpr*)

Description

Creates a Watcher object that monitors one or more simulation variables and triggers if the given condition is true. The condition string can be any valid CarMaker Real-time Expression (see CarMaker User's Guide section 'Real-time Expressions').

Example

```
Watcher = WatcherFactory.CreateConditionWatcher("Car.ay > 1.5")
```

DurationFactory

Creates Duration objects used to describe time frames to control the simulation and capture data. The DurationFactory can be accessed directly from the Testbench object:

```
durationfactory = Testbench.DurationFactory
```

<DurationFactory>.CreateTimeSpanDuration

Syntax

CreateTimeSpanDuration(float *timespan*)

Description

Creates a TimeSpanDuration object that can be used as a delay for the Start or Stop Trigger of a capture. The time span is given as a float value in seconds.

Example

```
TimeSpan = DurationFactory.CreateTimeSpanDuration(2.0)
```

CapturingFactory

Used to create MDFWriter objects that can be passed to a Capture to capture data to file. The CapturingFactory can be accessed directly from the Testbench object:

```
capturingfactory = Testbench.CapturingFactory
```

<CapturingFactory>.CreateCaptureResultMDFWriterByFileName

Syntax

CreateCaptureResultMDFWriterByFileName(string *filename*)

Description

Creates a CaptureResultMDFWriter object and sets its *filename* property. The CaptureResultMDFWriter object can be passed to a Capture to capture the configured data to the file specified with *filename*.



By default, this overwrites an existing file. To change this behavior, use the *Overwrite* property (see [section 2.2.5 'Example 5_CaptureToFile'](#)).

Example

```
CaptureWriter = CapturingFactory.CreateCaptureResultMDFWriterByFileName("MDFFile")
```

2.4.3 MAPort

The MAPort class provides most functionality needed to interact with CarMaker from Python code. The MAPort class is used to instantiate, configure and control the simulation program and interaction e.g. with reading and writing. After the MAPort class is instantiated with the [<MAPortFactory>.CreateMAPort\(\)](#) function, the MAPort is in the initial state *eDISCONNECTED*. MAPort objects can be created using the MAPortFactory function *CreateMAPort()*:

```
maport = MAPortFactory.CreateMAPort("DemoMAPort")
```

<MAPort>.LoadConfiguration

Syntax

LoadConfiguration(string *filename*)

Description

Loads the configuration parameters for the MAPort from an XML file specified with *filename* and returns an MAPortConfig object.

For CarMaker, the following parameters can be used in the configuration XML file.:

Parameter	Description
ProjectDir	Mandatory. Path to the project directory to be used for CarMaker.
Platform	Mandatory. Definition of the system CarMaker to run on. Valid values: • <i>office</i> • <i>xeno</i> • <i>dsrt</i> • <i>lvrt</i> • <i>nilrt</i> If <i>dsrt</i>, <i>lvrt</i> or <i>nilrt</i> are specified, CarMaker needs to be downloaded to the system via third-party tools before the Testbench is configured.

Parameter	Description
Hostname	Mandatory for real-time platforms, optional for office applications. Hostname or IP address to connect to. Specifies the hostname or IP of the target system CarMaker starts on to enable a connection. Hostname can also be supplied for office. In that case, CarMaker is started on the remote machine with the given hostname. Please note: In this case, all files for CarMaker need to be on the remote machine as well as on the local machine. In the case of dsrt platforms, a dSpace platform name can be specified here (e.g. SCALEXIO) to download to a SCALEXIO system registered on the host.
NICConfiguration	Only valid if Platform is <i>lvrt</i> or <i>nilrt</i>. The MAPort tries to configure an NI XIL Testbench with the given XML configuration file. If this parameter is given as a relative path, the path is handled as relative to the ProjectDir specified in the same configuration.
InstallPath	Optional. Custom CarMaker installation path.
CMPath	Optional. Custom path to the executable used for simulation.
CMArgs	Optional. CarMaker command line options.
DataPools	Optional. List of additional data pools.

Example command

```
config = MAPort.LoadConfiguration("Config.xml")
```

Example Config.xml

```
<?xml version="1.0" encoding="utf-8"?>
<PortConfigurations xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <MAPortConfig>
    <ProjectDir>%YourProject%</ProjectDir>
    <Platform>office</Platform>
  </MAPortConfig>
</PortConfigurations>
```

<MAPort>.Configure

Syntax

Configure(MAPortConfig *configuration*, boolean *force*)

Description

Configures the MAPort using the parameters previously loaded from the configuration file using [<MAPort>.LoadConfiguration\(\)](#). This starts up CarMaker if possible, or connects to an already running instance on a HIL system.

If the *force* parameter is set to *True*, the start-up is forced even if the Testbench has been configured before. If the start-up is not successful, the function raises a TestbenchPortException.

After a successful configuration, the MAPort switches to state *eSIMULATION_STOPPED*.

Example

```
MAPort.Configure(config)
```

<MAPort>.CreateCapture

Syntax

CreateCapture(string *name*)

Description

Creates a Capture and registers it with the given *name*, then returns the Capture object.

Example

```
Capture = MAPort.CreateCapture("captureTask")
```

<MAPort>.Read

Syntax

Read(string *variablename*)

Description

Reads a variable from the simulation and returns the value to the caller. All CarMaker variables are returned as floats.

Example

```
steerang = MAPort.Read("DM.Steer.Ang")
```

<MAPort>.Write

Syntax

Write(string *variablename*, float *value*, int *duration*)

Description

Writes the given *value* to the given variable (*variablename*) for the given number of cycles (*duration*).



One CarMaker cycle takes 1 ms. To overwrite the value for 1 s, a duration of 1000 cycles must be specified.

Example

```
MAPort.Write("DM.Steer.Ang", -2.0, 500)
```

<MAPort>.GetVariableInfo

Syntax

GetVariableInfo(string *variablename*)

Description

Returns a MAPortVariableInfo object containing *_Name*, *_Unit*, *_Type* and *_Readable/_Writeable* of the given variable. If the variable can not be found, the function returns *None*.

Example

```
info = MAPort.GetVariableInfo("DM.Steer.Ang")
```

<MAPort>.IsReadable

Syntax

IsReadable(string *variablename*)

Description

Returns *true* or *false* to indicate if the given variable can be read from CarMaker.

Example

```
if MAPort.IsReadable("DM.Steer.Ang"):
```

<MAPort>.IsWritable

Syntax

IsWritable(string *variablename*)

Description

Returns *true* or *false* to indicate if the given variable can be written to CarMaker.

Example

```
if MAPort.IsWritable("DM.Steer.Ang"):
```

<MAPort>.CheckVariableNames

Syntax

CheckVariableNames(string[] *variablenames*)

Description

Checks the existence of the given variables (*variablenames*) in the current CarMaker Test Run. Any variable that is not present in the DataDictionary is returned in the result list. This list is empty if all variables are valid.

Example

```
MAPort.CheckVariableNames(["Car.ax", "Car.ay", "Car.az"])
```

<MAPort>.StartSimulation

Syntax

<MAPort>.StartSimulation(string *testrun*)

Description

Starts the simulation with the given Test Run (*testrun*).

If no Test Run is passed to this function, the Test Run needs to be configured before calling this function using the [<MAPort>.DownloadParameterSets\(\)](#) function.

After a successful simulation start, the MAPort is in the state *eSIMULATION_RUNNING*.

If the start-up is not successful, this function raises a *TestbenchException*.

Example

```
MAPort.StartSimulation("Examples/BasicFunctions/Driver/BackAndForth")
```

<MAPort>.StopSimulation

Syntax

StopSimulation()

Description

Stops the currently running simulation and sets its state back to *eSIMULATION_STOPPED*.

Example

```
MAPort.StopSimulation()
```

<MAPort>.DownloadParameterSets

Syntax

DownloadParameterSets(string [] *filenames*)

Description

Loads the InfoFiles given in the *filenames* list one after another to parametrize CarMaker.

Supported InfoFiles:

- Test Run
- Vehicles
- Trailers
- Tires
- UserDriver InfoFiles



After the configuration is loaded using *DownloadParameterSets()*, the function *<MAPort>.StartSimulation()* can be called without a *testrun* parameter. If *StartSimulation()* is called with a *testrun* parameter, the parameters downloaded with *DownloadParameterSets()* are overwritten.

Example

```
1: MAPort.DownloadParameterSets(["Examples/BasicFunctions/Sensors/RadarRSI_Motorway",  
2: "Examples/DemoCar_SensorLidarRSI"  
3: "Examples/RT_235_55R18",  
4: "Examples/Caravan"])
```

Additional CarMaker convenience functions

<MAPort>.GetDDict

Syntax

GetDDict()

Description

Returns the entire current DataDictionary as a list of *MAPortVariableInfos* (with the parameters *_Name*, *_Unit*, *_DataType*, *_Readable*, *_Writeable*).



The Test Run to be accessed needs to be loaded into CarMaker first, so the DataDictionary is up-to-date.

Example

```
1: MAPort.StartSimulation("Examples/BasicFunctions/Sensors/RadarRSI_Motorway")
2: MAPort.StopSimulation()
3: ddict = MAPort.GetDDict()
```

<MAPort>.WaitForTime

Syntax

WaitForTime(float *simtime*, float *timeout*)

Description

Blocks the executing script until the given simulation time is reached without interfering with data capturing or other watchers. The optional time-out raises an exception if the simulation does not reach the given simulation time after *timeout* in seconds.

Example

```
MAPort.WaitForTime(10.0, 30.0)
```

<MAPort>.WaitForCondition

Syntax

WaitForCondition(string *condition*, float *timeout*)

Description

Blocks the executing script until the given condition is true without interfering with data capturing or other watchers. The optional time-out raises an exception if the condition is not true after *timeout* in seconds.

Example

```
MAPort.WaitForCondition("Car.av > 10")
```

<MAPort>.WaitForSimEnd

Syntax

WaitForSimEnd(float *timeout*)

Description

Blocks the executing script until the simulation is finished without interfering with data capturing or other watchers. The optional time-out raises an exception if the simulation is not finished after *timeout* in seconds.

Example

```
MAPort.WaitForSimEnd(120.0)
```

2.4.4 Capture

Capture functions enable to record data from the simulation program at runtime. Capture objects can be combined with Condition or Duration Watchers to capture only the specific frame of data you are interested in.

A new created Capture object starts in the state *eCONFIGURED*. A Capture object can be created using the *CreateCapture()* function of an MAPort object:

```
capture = MAPort.CreateCapture("captureTask")
```

The following functions are available as members of a Capture object.

<Capture>.SetStartTrigger

Syntax

SetStartTrigger(IWatcher *watcher*, IDuration *delay=None*)

Description

Sets a Trigger in the form of a Watcher object to start the capture. The watchers need to be created using the [WatcherFactory](#) and can be of type Duration ([<WatcherFactory>.CreateDurationWatcherByTimeSpan\(\)](#)) or Condition ([<WatcherFactory>.CreateConditionWatcher\(\)](#)).

In the second argument, the user can specify an optional *delay* after the Trigger is fired. Use a negative value for an optional delay before the Trigger is fired. Negative delay is only possible if the delay is not longer than the buffer size.

If no start trigger is set, the capture is stopped when the next simulation ends.

Example

```
Capture.SetStartTrigger(watcher, delay)
```

<Capture>.SetStopTrigger

Syntax

SetStopTrigger(IWatcher *watcher*, IDuration *delay=None*)

Description

Sets a Trigger in the form of a Watcher object to stop the capture. The watchers need to be created using the [WatcherFactory](#) and can be of type Duration ([<WatcherFactory>.CreateDurationWatcherByTimeSpan\(\)](#)) or Condition ([<WatcherFactory>.CreateConditionWatcher\(\)](#)).

In the second argument, the user can specify an optional *delay* after the Trigger is fired. Use a negative value for an optional delay before the Trigger is fired. Negative delay is only possible if the delay is not longer than the buffer size. Negative delays can not be used when capturing data to a file.

If no stop trigger is set, the capture is stopped when the next simulation ends.

Example

```
Capture.SetStopTrigger(watcher, delay)
```

<Capture>.Start

Syntax

Start(ICaptureResultWriter *writer=None*)

Description

Sets the *State* of the Capture to *eACTIVATED*. The Capture is scheduled and started when the next simulation starts or the Start Trigger is activated. This also resets the `Capture.CaptureResult`.

To capture the data to a file, an optional *writer* parameter can be passed to this function.

Example

```
Capture.Start(writer)
```

<Capture>.Stop

Syntax

Stop()

Description

Stops the capture if it has not finished yet and sets the state back to *eCONFIGURED*.

Example

```
Capture.Stop()
```

<Capture>.Fetch

Syntax

Fetch(bool *whenFinished*)

Description

Returns captured data in the form of a [CaptureResult](#).

The function can only be called if the capture is in State *eACTIVATE*, *eRUNNING* or *eFINISHED*.

If *Fetch()* is called with the *whenFinished* parameter set to *False*, the behavior depends on the *CaptureState*. If it is *eFINISHED*, *Fetch()* returns all captured data in a single *CaptureResult*. The function can also be called repeatedly while the simulation is still in state *eRUNNING*. In this case, *Fetch()* returns all data captured since the previous *Fetch()* call (or the start of the Capture).

If *Fetch()* is called with the *whenFinished* parameter set to *True*, the call waits (and blocks the script) until the Capture is finished, either through the Stop Trigger or because the simulation is finished, and then returns the *CaptureResult*.

If a filewriter is defined to capture to file, *Fetch()* raises an exception. The same applies to *Fetch()* calls in the *CaptureState* *eCONFIGURED* or if *Fetch(True)* is called without setting a Stop Trigger first.

Example

```
result = Capture.Fetch(True)
```

<Capture>.Variables

Description

Sets variable names the capture subscribes to.

Example

```
Capture.Variables = ["Car.ax", "Car.ay", "Car.az"]
```

<Capture>.Downsampling

Description

The rate of downsampling the variables are captured with, e.g. a downsampling of 50 means that every 50th value is captured (translating to a data rate of 20 Hz).

By default, Captures are initialized with a downsampling of 10 (or 100 Hz).

Example

```
Capture.Downsampling = 50
```



<Capture>.MinBuffersize

Description

For the CarMaker MAPort, the buffer size of a capture determines the number of data points buffered before they are overwritten.

With a sampling rate of 1000 Hz and a downsampling rate of 50, 20 samples (or data points) per second are created. Having a buffer size of 10000 data points, the buffer is filled after 500 seconds:

$$\frac{\text{Buffersize} \times \text{Downsampling}}{\text{Samplingrate}} = \frac{10000 \times 50}{1000\text{Hz}} = 500\text{s}$$

To capture more data, either expand the buffer size or fetch (<Capture>.Fetch()) the data from the buffer multiple times during simulation.

Example

```
Capture.MinBuffersize = 10000
```

2.4.5 CaptureResult

A *CaptureResult* contains the data collected during a simulation in the form of SignalGroupValue objects.

SignalGroupValue objects consist of an XVector (the time stamps in case of CarMaker) and the YVector values of the variables set for the capture. All vectors are VectorValue objects that contain the variable values as an array and a `_Count` parameter indicating the size of the vector. CaptureResults can be retrieved from a Capture object by calling the <Capture>.Fetch() function or accessing the CaptureResult parameter after the capture has finished:

```
1: if Capture.State is CaptureState.eFINISHED:
2:     captureresult = Capture.CaptureResult
3:     captureresult = Capture.Fetch(True)
```

The following functions can be called as members of a CaptureResult object.

<CaptureResult>.ExtractSignalValue

Syntax

ExtractSignalValue(string *variablename*)

Description

Returns the data captured for the given variable as a SignalValue consisting of the data vector and a time vector.

Example

```
1: signal = CaptureResult.ExtractSignalValue("Car.ax")
2: values = signal.FcnValues.Value
3: timestamps = signal.XVector.Value
```

<CaptureResult>.GetSignalGroupValues

Syntax

GetSignalGroupValues()

Description

Returns the entire Capture as a SignalGroupValue object.

Example

```
signalgroup = CaptureResult.GetSignalGroupValues()
```

<CaptureResult>.GetVariableNames

Syntax

GetVariableNames()

Description

Returns a string array of the variable names contained in the capture.

Example

```
variables = CaptureResult.GetVariableNames()
```